

6. 임베딩 (Word2Vec ~)

Word2Vec

단어 벡터화

CBoW(Continuous Bag of Words)

Skip-gram

계층적 소프트맥스(Hierarchical Softmax)

네거티브 샘플링

모델 실습 : Skip-gram

모델 실습 : Gensim

fastText

모델 실습

순환 신경망 (Recurrent Neural Network, RNN)

순환 신경망

일대다 구조 (One-to-Many)

다대일 구조 (Many-to-One)

다대다 구조 (Many-to-Many)

양방향 순환 신경망 (Bidirectional Recurrent Neural Network, BiRNN)

다중 순환 신경망 (Stacked Recurrent Neural Network)

순환 신경망 클래스

장단기 메모리 (Long Short-Term Memory, LSTM)

장단기 메모리 구조

장단기 메모리 클래스

모델 실습

합성곱 신경망 (Convolutional Neural Network, CNN)

합성곱 계층

필터 (Filter)

패딩 (Padding)

간격 (Stride)

채널

팽창 (Dilation)

합성곱 계층 클래스

활성화 맵 (Activation Map)

풀링

완전 연결 계층 (Fully Connected Layer, FC)

모델 실습

텍스트용 CNN : 1차원 합성곱 (1D Convolution)

Word2Vec

- 단어 간 유사성을 측정하기 위해 분포 가설(distributional hypothesis)을 기반으로 개발된 구글에서 공개한 임베딩 모델
 - 분포 가설 : 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정.
단어 간 동시 발생(co-occurrence) 확률 분포를 이용해 단어 간의 유사성을 측정.
- 분포 가설을 통해 단어의 분산 표현(Distributed Representation)을 학습할 수 있음
 - 분산 표현 : 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
- 분포 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남 → 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 가짐

- 이러한 방법으로 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계를 극복함
- 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화 해 단어의 의미 정보를 효과적으로 표현 가능

단어 벡터화

단어를 벡터화하는 방법

- 희소 표현(sparse representation)

표 6.3 단어의 희소 표현

소	0	1	0	0	0
일고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- e.g., one-hot encoding, TF-IDF 등 빈도 기반 방법
- 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비 발생
- 단어의 유사성을 반영하지 못하고, 벡터 간의 유사성을 계산하는 데도 많은 비용이 발생함
- 밀집 표현(dense representation)

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
일고	-0.2893	0.3814	...	-0.1492	-0.2814
외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- e.g., Word2Vec와 같은 밀집 표현
- 단어를 고정된 크기의 실수 벡터로 표현 → 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 X
- 벡터 공간상에서 단어 간의 거리를 효과적으로 계산 가능
- 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적인 공간 활용 가능
- 단어를 벡터화하기 때문에, 단어의 의미 비교 가능
- 밀집 표현된 벡터 = 단어 임베딩 벡터(Word Embedding Vector)

Word2Vec은 밀집 표현을 위해 CBoW, Skip-gram이라는 두 가지 방법 사용

CBoW(Continuous Bag of Words)

- 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법
 - 중심 단어(Center Word) : 예측해야 할 단어
 - 주변 단어(Context Word) : 예측에 사용되는 단어들
 - Window : 중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지 정하는 범위
 - Sliding Window : 학습을 위해 윈도우를 이동해 가며 학습하는 방법

- CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습할 수 있다.

학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은) 모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두) 밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에) 일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

하나의 입력 문장에서 윈도우 크기가 2일 때의 학습 데이터 구성

- 학습 데이터는 (주변 단어 | 중심 단어)로 구성됨 → 대량의 말뭉치에서 효율적으로 단어의 분산 표현 학습 가능
- 얻어진 학습 데이터는 아래 그림과 같은 구조의 인공 신경망을 학습하는 데 사용됨

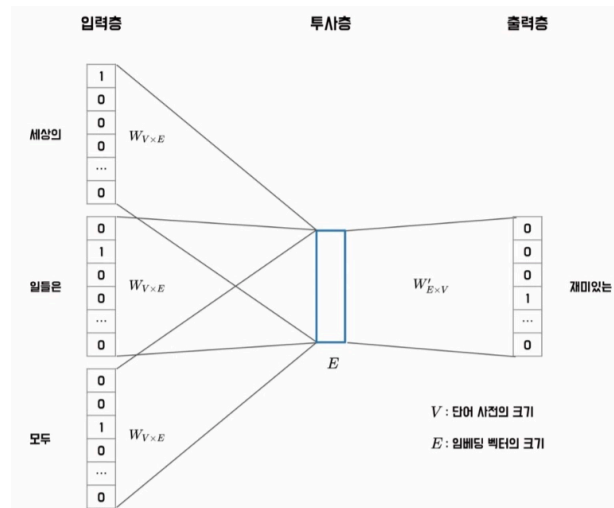


그림 6.5 CBoW 모델 구조

- 투사층 = Lookup table. 행렬 곱이 아니라 인덱스로 임베딩을 가져오는 구조
- 주변 단어 여러 개 → 평균 → 하나의 중심 단어 예측 (Many-to-One)
- 임베딩 차원 E , 어휘 크기 V 가 핵심 하이퍼파라미터

CBoW 처리 흐름

1. 입력 단어를 원-핫 벡터로 표현
2. 투사층(Projection Layer) 통과 → 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 룩업테이블(LUT)에서 꺼냄
3. 입력된 모든 단어의 임베딩 벡터를 평균 냄 (e.g. $V_{\text{세상의}}, V_{\text{일들은}}, V_{\text{모두}} \rightarrow \text{평균}$)
4. 평균 벡터 $\times$ 가중치 행렬 $W'_{E \times V} \rightarrow V$ 크기의 벡터 생성
5. 소프트맥스 함수로 중심 단어 예측

Skip-gram

- CBoW와 반대로, 중심 단어 하나로 주변 단어들을 예측하는 모델. (One-to-Many)

입력 문장	학습 데이터
(세상의 재미있는 일들은) 모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두) 밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에) 일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의 (재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는 (일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은 (모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

- CBoW와의 핵심 차이
 - CBoW : 원도 하나 → 학습 데이터 하나
 - Skip-gram : 원도 하나 → 학습 데이터 여러 개 (중심 단어 × 각 주변 단어쌍)
 - 그래서 학습 데이터량이 더 많고, 일반적으로 CBoW보다 성능이 뛰어남
 - 희귀 단어 학습에도 더 강함

처리 흐름

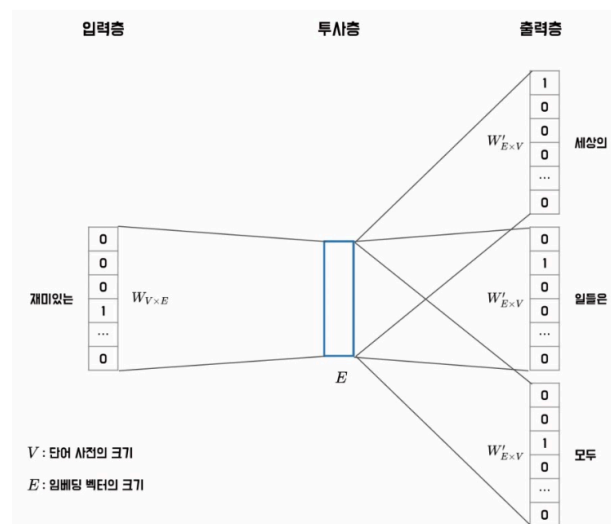


그림 6.7 Skip-gram 모델 구조

1. 중심 단어를 원-핫 벡터로 입력
2. 투사층(LUT)에서 해당 단어의 임베딩 벡터 추출
3. 임베딩 $\times W'_{E \times V} \rightarrow V$ 크기의 벡터
4. 소프트맥스로 각 주변 단어를 개별 예측

단점 및 해결책

- 소프트맥스가 어휘 전체(V)를 대상으로 내적 연산을 하기 때문에, 말뭉치가 커질수록 학습 속도가 급격히 느려짐.
- 해결책 두 가지

- 계층적 소프트맥스(Hierarchical Softmax)
- 네거티브 샘플링(Negative Sampling)

계층적 소프트맥스(Hierarchical Softmax)

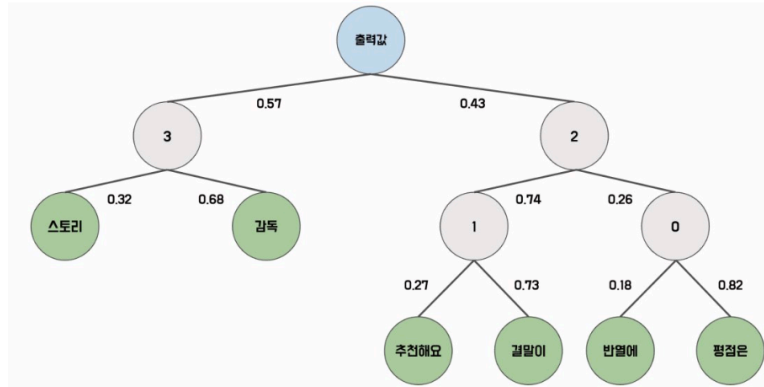


그림 6.8 계층적 소프트맥스 구조

- 일반 소프트맥스의 $O(V)$ 연산 비용을 줄이기 위한 근사 기법.
- 출력층을 이진 트리(Binary Tree)로 구성. 단어를 트리의 잎 노드(Leaf Node)에 배치하고, 루트에서 해당 잎까지의 경로 노드들만 계산함

트리 구성 규칙

- 자주 등장하는 단어 → 상위 노드 (경로 짧음 → 연산 적음)
- 드물게 등장하는 단어 → 하위 노드 (경로 길지만 등장 빈도 자체가 낮음)

확률 계산 방식

- 각 노드는 학습 가능한 벡터를 가짐
- 입력값과 노드 벡터의 내적 → 시그모이드 → 분기 확률 결정
- 특정 단어의 확률 = 루트에서 해당 잎까지 경로의 확률을 모두 곱함
- 예: '추천해요' = $0.43 \times 0.74 \times 0.27 = \text{약 } 0.086$

학습 시 이점 해당 단어의 경로 위 노드 벡터만 업데이트하면 됨. 전체 어휘를 건드릴 필요 없음.

시간 복잡도

- 일반 소프트맥스 : $O(V)$
- 계층적 소프트맥스 : $O(\log_2 V)$

네거티브 샘플링

- 전체 어휘 V 에 대한 소프트맥스 연산 대신, 일부 단어만 샘플링해서 연산량을 줄이는 기법.
- 문제를 다중 분류(어떤 단어인지 맞추기)에서 이진 분류(이 단어 쌍이 진짜인지 가짜인지)로 바꿈.
 - 실제 학습 데이터에서 나온 쌍 → 레이블 1 (Positive)
 - 랜덤 샘플링된 가짜 쌍 → 레이블 0 (Negative)

샘플링 방법

- 학습 윈도에 등장하지 않는 단어를 n개 추출 (보통 5~20개)
- 추출 확률: $P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0} f(w_j)^{0.75}}$
- 0.75 제곱을 쓰는 이유: 빈도 차이를 완화해 희귀 단어가 너무 적게 뽑히는 문제를 보정 (실험적으로 최적화된 값)

학습 방식

- 입력 단어 임베딩과 후보 단어 임베딩의 내적 → 시그모이드 → 확률값
- 레이블 1이면 확률값이 높아지도록, 레이블 0이면 낮아지도록 가중치 업데이트

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

입력 데이터	출력 데이터
재미있는, 세상의	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 결어기다	0
재미있는, 주심시오	0
재미있는, 미리	0
재미있는, 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

- 로지스틱 회귀 모델 구조와 동일

일반 소프트맥스 Word2Vec과의 차이

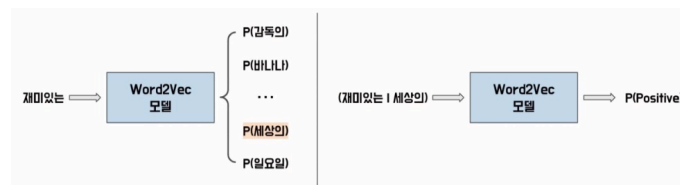


그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

- 일반: 단어 하나 입력 → 전체 어휘에 대한 확률 분포 출력
- 네거티브 샘플링: 단어 쌍 입력 → 그 쌍이 진짜일 확률 하나 출력

모델 실습 : Skip-gram

```
# 임베딩 클래스
embedding = torch.nn.Embedding(
    num_embeddings, # 임베딩 수 = 이산 변수의 개수. 단어 사전의 크기를 의미
    embedding_dim, # 임베딩 차원 = 임베딩 벡터의 차원 수. 임베딩 벡터의 크기를 의미
    padding_idx=None, # 패딩 인덱스. 패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정
    max_norm=None, # 임베딩 벡터의 최대 크기 지정
    norm_type=2.0 # 임베딩 벡터의 크기를 제한하는 방법 선택. default=2(L2 regularization 사용). 1 선택시 L1 사용
)
```

```
# 기본 Skip-gram 클래스
from torch import nn

class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim
        )
        self.linear = nn.Linear(
            in_features=self.embedding_dim,
            out_features=vocab_size
        )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output
```

```
# 영화 리뷰 데이터셋 전처리
import pandas as pd
from Korpora import Korpora
from konlpy.tag import Okt

corpus = Korpora.load("nsmc")
corpus = pd.DataFrame(corpus.test) # testset 불러오기

tokenizer = Okt() # 형태소 추출
tokens = [tokenizer.morphs(review) for review in corpus.text]
print(tokens[:3])
```

▼ 실행 결과

```
[nsmc] download ratings_train.txt: 14.6MB [00:06, 2.12MB/s]
[nsmc] download ratings_test.txt: 4.90MB [00:02, 2.21MB/s]
[['곧', 'ㅋ'], ['GDNTOPCLASSINTHECLUB'], ['뭐', '야', '이', '평점', '들', '은', '....', '나쁜진', '않지만', '10', '점', '짜', '리', '는', '더', '더욱', '아니잖아']]
```

```
# 단어 사전 구축
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens): # 단어 사전 구축
    ...

    n_vocab : 구축할 단어 사전의 크기
    special_tokens : 특별한 의미를 갖는 토큰들
```

```

...
counter = Counter()
for tokens in corpus:
    counter.update(tokens)
vocab = special_tokens
for token, count in counter.most_common(n_vocab):
    vocab.append(token)
return vocab

vocab = build_vocab(corpus=tokens, n_vocab=5000, special_tokens=["<unk>"])
# 단어 사전 내에 없는 모든 단어는 <unk> 토큰으로 대체
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))

```

▼ 실행 결과

```

['<unk>', '.', '이', '영화', '의', '...', '가', '에', '...', '을']
5001

```

special_token이 1개이므로 총 5,001개로 구성

```

# Skip-gram의 단어 쌍 추출
def get_word_pairs(tokens, window_size): # 토큰 입력받아 skip-gram 모델의 입력
    # 데이터로 사용가능하게 전처리
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            center_word = sentence[idx]
            context_words = sentence[window_start:idx] + sentence[idx+1:window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs

word_pairs = get_word_pairs(tokens, window_size=2)
print(word_pairs[:5])

```

▼ 실행 결과

```

[['글', 'ㅋ'], ['ㅋ', '글'], ['뭐', '야'], ['뭐', '이'], ['야', '뭐']]

```



```
# 인덱스 쌍 변환
def get_index_pairs(word_pairs, token_to_id): # get_word_pairs 함수에서 생성
    된 단어 쌍을 토큰 인덱스 쌍으로 변환
    pairs = []
    unk_index = token_to_id["<unk>"]
    for word_pair in word_pairs:
        center_word, context_word = word_pair
        center_index = token_to_id.get(center_word, unk_index)
        context_index = token_to_id.get(context_word, unk_index)
        pairs.append([center_index, context_index])
    return pairs

index_pairs = get_index_pairs(word_pairs, token_to_id)
print(index_pairs[:5])
```

▼ 실행 결과

```
[[595, 100], [100, 595], [77, 176], [77, 2], [176, 77]]
```

```
# 데이터로더 적용
import torch
from torch.utils.data import TensorDataset, DataLoader

index_pairs = torch.tensor(index_pairs)
center_indexes = index_pairs[:, 0]
context_indexes = index_pairs[:, 1]

dataset = TensorDataset(center_indexes, context_indexes)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

```
# Skip-gram 모델 준비 작업
from torch import optim

device = "mps" if torch.backends.mps.is_available() else "cpu"
word2vec = VanillaSkipgram(vocab_size=len(token_to_id), embedding_dim=12
8).to(device)
criterion = nn.CrossEntropyLoss().to(device) # CE는 내부적으로 소프트맥스 연산
을 수행하므로 신경망의 출력값을 후처리 없이 사용 가능
optimizer = optim.SGD(word2vec.parameters(), lr=0.1)
```

```
# 모델 학습
for epoch in range(10):
    cost = 0.0
    for input_ids, target_ids in dataloader:
```

```

input_ids = input_ids.to(device)
target_ids = target_ids.to(device)

logits = word2vec(input_ids)
loss = criterion(logits, target_ids)

optimizer.zero_grad()
loss.backward()
optimizer.step()

cost += loss

cost = cost / len(dataloader)
print(f"Epoch : {epoch+1:4d}, Cost : {cost:.3f}")

```

▼ 실행 결과

```

Epoch :   1, Cost : 6.198
Epoch :   2, Cost : 5.981
Epoch :   3, Cost : 5.931
Epoch :   4, Cost : 5.901
Epoch :   5, Cost : 5.879
Epoch :   6, Cost : 5.862
Epoch :   7, Cost : 5.847
Epoch :   8, Cost : 5.834
Epoch :   9, Cost : 5.823
Epoch :  10, Cost : 5.812

```

```

# 임베딩 값 추출
token_to_embedding = dict()
embedding_matrix = word2vec.embedding.weight.detach().cpu().numpy()

for word, embedding in zip(vocab, embedding_matrix):
    token_to_embedding[word] = embedding

index = 30
token = vocab[30]
token_embedding = token_to_embedding[token]
print(token)
print(token_embedding)

```

▼ 실행 결과

연기

```
[ 0.2941101 -0.21780542 -0.68241346  0.5940522  0.5118865  0.00802056
 0.5786206 -0.28015885  0.8484222  1.0802952 -0.5618869 -0.23117726
-0.5439373  0.19624062  0.61089283  0.8487472  0.64098465 -0.6874315
 0.41905674  0.1775276 -1.3241575  0.23154491 -0.3352039  1.3502827
-0.20423089  0.697836  1.5268756 -0.09745841 -1.8211484  0.96218634
-0.23088011 -2.6053748 -0.27091095  0.4508879 -0.09909813 -0.7957805
 1.3402754  1.6122129  0.33817104 -0.25272655  1.1563859 -0.5401144
-0.00696711  0.8275276 -1.0016899 -0.49301645 -0.07436889  1.0973963
 2.268987  0.04290099 -0.06325626 -0.00762811 -0.20286708  0.3268943
 0.68802476  0.64360315  0.6943373 -0.44744438 -0.35450542 -1.2695695
 0.2549332  0.9296391 -1.566026 -0.752718 -0.36082146  0.96778595
 1.0098866 -0.28831264  2.3126929  0.06721981  0.9791606  2.8674133
 1.3816512  0.6861534 -0.6619561  1.0058658 -1.3813322 -0.0936224
 0.8211212  0.682402 -0.6760864  0.3429375  0.13169025 -0.47543103
 1.9820248 -0.7226095  0.47800124 -0.05153184 -1.9070531  0.89037174
-0.19454896  0.10204849  0.11111134  1.9108722 -0.9387047  0.18551114
-0.74607664  0.13729829  0.4043284 -0.62954473  0.34558046  0.45964032
 1.7929813  0.28944394  0.0032196  0.8884027 -0.58750844  0.6436494
-0.8436518  0.78891784  1.7259849  0.22575763  1.0401324  0.8287211
-0.21832584 -1.5446435 -1.2155948  1.7934461 -0.7603587 -0.08439486
-0.11975027  0.72055376 -0.43386945  0.2884346  0.5434663 -1.1385833
-0.6614664 -0.796688 ]
```

- Word2Vec 모델의 임베딩 행렬 이용해 각 단어의 임베딩 값 매핑하고, 인덱스 30 값의 단어와 임베딩 값 출력
→ 단어 간 유사도 확인 가능
- 임베딩의 유사도 측정 — 코사인 유사도(Cosine Similarity)가 가장 일반적으로 사용됨
 - 두 벡터 간의 각도를 이용해 유사도 계산
 - 두 벡터가 유사할수록 값이 1에 가까워짐. (다를수록 0에 가까움)
 - 계산 : 두 벡터의 내적을 벡터의 크기(유클리드 norm)의 곱으로 나누어 계산

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

```
# 단어 임베딩 유사도 계산
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b): # 코사인 유사도 계산
    '''
    a : 임베딩 토큰
    b : 임베딩 행렬
    '''
    cosine = np.dot(b, a) / (norm(b, axis=1) * norm(a))
    return cosine

def top_n_index(cosine_matrix, n): # 유사도 행렬을 내림차순으로 정렬해 어떤 단어가 가장 가까운 단어인지 반환
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1] # 입력 단어도 단어 사전에 포함 -> 두번째 가까운 단어부터 반환
```

```

return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도 : {cosine_matrix[index]:.4f}")

```

▼ 실행 결과

```

연기와 가장 유사한 5 개 단어
미스캐스팅 - 유사도 : 0.3311
재미있었는데 - 유사도 : 0.3088
영 - 유사도 : 0.2945
고양이 - 유사도 : 0.2913
위주 - 유사도 : 0.2738

```

모델 실습 : Gensim

Gensim : Word2Vec 같은 NLP 모델을 쉽고 빠르게 구성할 수 있는 파이썬 라이브러리

주요 특징

- 메모리 효율적 설계 → 대규모 텍스트 데이터도 처리 가능
- 학습된 모델 저장/불러오기 지원
- 유사 단어 찾기 등 유사도 관련 기능 내장
- 내부적으로 Cython(C++ 기반) 사용 → 병렬 처리, 네거티브 샘플링 등을 파이토치보다 훨씬 빠르게 실행

설치

```
pip install gensim
```

쓰는 이유

- 직접 구현한 Word2Vec은 데이터가 적어도 학습이 느림
- Gensim은 네거티브 샘플링 등 최적화 기법이 이미 내장되어 있어 실무에서 빠르게 쓸 수 있음

```

# Word2Vec 클래스
word2vec = gensim.models.Word2Vec(
    sentences=None, # 입력 문장(모델의 학습 데이터). 토큰 리스트로 표현됨
    corpus_file=None, # 말뭉치 파일. 학습 데이터를 파일로 입력할 때 파일의 경로(입
    력 문장 대신 사용가능)
    vector_size=100, # 학습할 임베딩 벡터의 크기. 임베딩 차원 수 설정함.
    alpha=0.025, # Word2Vec 모델의 learning_rate
    window=5, # 학습 데이터를 생성할 윈도우의 크기
    min_count=5, # 학습에 사용할 단어의 최소 빈도
    workers=3, # 빠른 학습을 위해 병렬 학습할 스레드 수
    sg=0, # Skip-gram 모델 사용여부 설정. 0이면 CBoW 모델 사용

```

```

hs=0, # 계층적 소프트맥스 사용여부 설정
cbow_mean=1, # CBoW 모델로 구성할 때 사용되는 하이퍼파라미터. 중심 단어, 주변
단어를 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부 설정.
negative=5, # 네거티브 샘플링의 단어 수
ns_exponent=0.75, # 네거티브 샘플링 확률 지수
max_final_vocab=None, # 단어 사전의 최대 크기
epochs=5,
batch_words=10000 # 몇 개의 단어로 학습 배치 구성할지
)

```

```

# Word2Vec 모델 학습
from gensim.models import Word2Vec

word2vec = Word2Vec(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=1,
    sg=1,
    epochs=3,
    max_final_vocab=10000
)

# word2vec.save("../models/word2vec.model")
# word2vec = Word2Vec.load("../models/word2vec.model")

```

```

# 임베딩 추출 및 유사도 계산
word = "연기"
print(word2vec.wv[word]) # wv 속성 : 학습된 단어 벡터 모델을 포함한 Word2VecKeye
dVectors 객체 반환
print(word2vec.wv.most_similar(word, topn=5)) # 주어진 단어에 대해 가장 유사한
단어 찾는 메서드
print(word2vec.wv.similarity(w1=word, w2="연기력")) # 두 단어 간의 유사도 계산
하는 메서드

```

▼ 실행 결과

```

[-0.02667657 -0.34862036 0.41338915 0.4686809 -0.04125457 0.00670285
 0.0879682 -0.03342802 -0.4797446 0.33667064 -0.09459545 -0.2892723
 -0.20146132 -0.21549569 -0.27059954 0.14191194 -0.2172086 0.48347062
 -0.19045451 0.50182915 0.6826414 0.61021006 -0.3031125 -0.12059221
 -0.14227532 -0.05954101 -0.30990505 0.2567402 0.19191884 -0.22100559
 -0.31433555 0.29907176 0.36993414 -0.11773805 -0.15907387 -0.4389728
 0.24327785 0.00516449 -0.04664818 -0.25932115 -0.21387108 0.4042731
 -0.20903711 -0.4153576 -0.14224282 0.16677971 -0.11068069 -0.2814738
 0.12248484 0.15013784 0.7595337 0.42844963 0.02742598 0.34533462
 -0.57871413 -0.1728636 0.05606428 0.26987085 -0.29829356 0.21758617
 -0.12628514 -0.14511463 0.132431 0.1063849 -0.22754714 0.00590633
 0.00470865 0.15111609 0.25323346 -0.13863517 -0.2770997 -0.3826897
 -0.2526759 0.23234917 -0.2646175 0.1410449 -0.2522412 -0.41739193
 -0.11078971 0.2903659 -0.02431638 -0.27307373 0.09937136 0.79197073
 0.19314912 0.33529645 0.12426993 -0.44900978 0.0062007 -0.07479651
 -0.05572437 -0.11050379 0.13680045 -0.06583385 0.39476216 0.0512414
 0.06360972 0.00758856 -0.15998213 -0.30451223 -0.2350494 -0.1788089
 0.21290714 -0.4593501 -0.00359314 -0.0639808 0.5068051 -0.00294602
 -0.04112155 -0.3086968 0.2730993 -0.01477829 -0.02311215 0.25633118
 0.1258802 -0.28123057 0.32171932 0.33645558 -0.03274422 0.38541088
 0.01209893 -0.21041782 0.08537282 -0.06351171 -0.14970371 0.07355148
 -0.5745575 -0.2791181 ]
[('연기력', 0.7989827394485474), ('캐스팅', 0.7349828481674194), ('연기자', 0.7166949510574341),
0.79898274

```

Word2Vec 한계

- 형태학적 특징 미반영
 - Word2Vec은 분포 가설 기반으로 단어의 의미는 잘 학습하지만, 단어의 내부 구조(형태)는 학습하지 못함.
- 한국어에서 특히 문제가 되는 이유
 - 한국어는 교착어로, 하나의 어근에 접사·조사가 결합해 다양한 형태가 만들어짐. 예를 들어 "먹다", "먹고", "먹어서"는 같은 어근이지만 Word2Vec은 이를 완전히 다른 단어로 취급.
- 결과적으로 발생하는 문제
 - OOV(Out-Of-Vocabulary) 증가 — 학습 데이터에 없는 형태의 단어가 등장하면 처리 불가.

fastText

- 2015년 Meta FAIR에서 개발한 오픈소스 임베딩 모델. Word2Vec의 형태학적 한계를 극복하기 위해 하위 단어(Subword) 단위로 학습.

Word2Vec과의 핵심 차이

- Word2Vec은 단어 전체를 하나의 단위로 학습
- fastText는 단어를 N-gram으로 쪼개 하위 단어 단위로 학습 → 형태 정보 반영 가능

처리 흐름 (e.g. "서울특별시")

입력 단어	〈 〉 더하기	N-gram 분해	전체 단어 추가
서울특별시	〈서울특별시〉	〈서울 서울특 특별 특별시 별시〉	〈서울 서울특 서울특 특별 특별 특별 별시 별시〉 〈서울특별시〉

그림 6.11 fastText의 하위 단어 집합

1. 양 끝에 〈, 〉 추가 → 〈서울특별시〉

2. N-gram으로 분해 → 〈서울, 서울특, 울특별, 특별시, 별시〉
3. 분해된 하위 단어 집합 + 전체 토큰 자체도 포함
4. 각 하위 단어가 고유한 임베딩 벡터를 가짐
5. 모든 하위 단어 벡터를 합산 → 최종 단어 임베딩

핵심 장점

- 같은 하위 단어를 공유하는 단어끼리 유사한 임베딩을 가짐
- OOV 처리 가능 — 말뭉치에 없는 단어도 하위 단어가 겹치면 임베딩 계산 가능 (예: '개인정보보호법'을 본 적 없어도 '개인택시', '정보처리기사' 등의 하위 단어로 추론)

모델 실습

- fastText 모델도 CBoW와 Skip-gram으로 구성되며 네거티브 샘플링 기법을 사용해 학습
- 하위 단어로 학습 (Word2Vec 모델은 기본 단위로 모델을 학습함)

```
fasttext = gensim.models.FastText(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window=5,
    min_count=5,
    workers=3,
    sg=0,
    hs=0,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000,
    min_n=3, # 사용할 N-gram의 최솟값
    max_n=6 # 사용할 N-gram의 최댓값
)
```

```
# KorNLI 데이터셋 전처리
from Korpora import Korpora

corpus = Korpora.load("kornli")
corpus_texts = corpus.get_all_texts() + corpus.get_all_pairs()
tokens = [sentence.split() for sentence in corpus_texts]

print(tokens[:3])
```

▼ 실행 결과

```
[kornli] download multilinli.train.ko.tsv: 83.6MB [00:36, 2.29MB/s]
[kornli] download snli_1.0_train.ko.tsv: 78.5MB [00:38, 2.06MB/s]
[kornli] download xnli.dev.ko.tsv: 516kB [00:00, 1.50MB/s]
[kornli] download xnli.test.ko.tsv: 1.04MB [00:00, 1.70MB/s]
[['개념적으로', '크림', '스키밍은', '제품과', '지리라는', '두', '가지', '기본', '차원을', '가지고', '있다.']]
```

```
# fastText 모델 실습
from gensim.models import FastText

fastText = FastText(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=5,
    sg=1,
    epochs=3,
    min_n=2,
    max_n=6
)

# fastText.save("../models/fastText.model")
# fastText = FastText.load("../models/fastText.model")
```

```
# fastText OOV 처리
oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))
```

▼ 실행 결과

```
False
[('사랑해', 0.90847247838974), ('사랑한', 0.8699836730957031), ('사랑', 0.8699661493301392), ('사랑해서', 0.8511850833892822), ('사랑해.', 0.8417598009109497)]
```

순환 신경망 (Recurrent Neural Network, RNN)

순환 신경망

- 순서가 있는 연속형 데이터(Sequence data)를 처리하기 위한 신경망. 이전 시점의 정보를 현재 시점에 활용하는 구조
- 왜 연속형 데이터가 필요한가?
자연어는 단어 순서에 따라 의미가 결정되고, t번째 단어는 t-1번째까지의 단어에 영향을 받음. 단순 피드포워드 신경망으로는 이 순서 정보를 반영할 수 없음.

핵심 구조

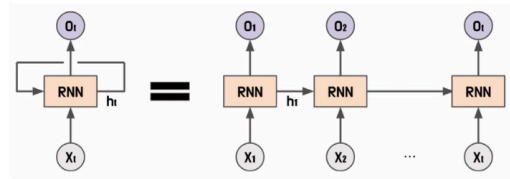


그림 6.12 순환 신경망의 셀

- 셀(Cell): 각 시점의 입력을 받아 은닉 상태와 출력값을 계산하는 단위
- 은닉 상태(Hidden state): 과거 정보를 압축해 다음 시점으로 전달하는 벡터
- 같은 셀이 시점마다 반복 적용됨 (가중치 공유)

수식

- 은닉 상태

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- W_{hh} : 이전 은닉 상태에 대한 가중치
- W_{xh} : 현재 입력값에 대한 가중치
- 출력값

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- W_{hy} : 현재 은닉 상태에 대한 가중치

모델 구조 종류

- 일대일, 일대다, 다대일, 다대다로 설계 가능. 태스크에 따라 구조를 선택함

일대다 구조 (One-to-Many)

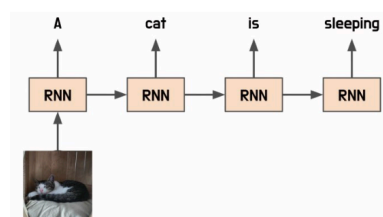


그림 6.13 순환 신경망의 일대다 구조

- 하나의 입력 시퀀스 → 여러 개의 출력값을 생성하는 RNN 구조

활용 예시

- 품사 태깅: 문장(입력) → 각 단어의 품사(출력)
- 이미지 캡셔닝: 이미지(입력) → 설명 문장(출력)

구현 시 주의점

- 출력 시퀀스의 길이를 미리 알아야 함. 따라서 입력 시퀀스를 처리하면서 출력 길이를 예측하는 모델을 함께 구현해야 함.

다대일 구조 (Many-to-One)

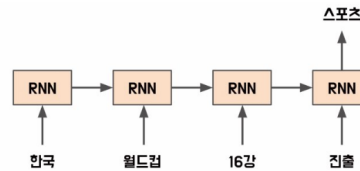


그림 6.14 순환 신경망의 다대일 구조

- 여러 개의 입력 시퀀스 → 하나의 출력값을 생성하는 RNN 구조

활용 예시

- 감성 분류(Sentiment Analysis): 문장(입력) → 긍정/부정(출력)
- 문장 분류: 문장(입력) → 카테고리(출력)
- 자연어 추론(NLI): 두 문장(입력) → 관계(출력)

특징

- 마지막 시점의 은닉 상태만 출력으로 사용
- 전체 시퀀스의 정보가 하나의 벡터로 압축됨

다대다 구조 (Many-to-Many)

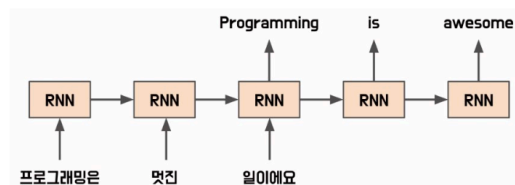


그림 6.15 순환 신경망의 다대다 구조

- 여러 개의 입력 시퀀스 → 여러 개의 출력 시퀀스를 생성하는 RNN 구조

활용 예시

- 번역기: 입력 문장 → 번역된 출력 문장
- 음성 인식: 음성 신호 → 텍스트 문장

입출력 길이가 다를 때?

- 입력과 출력 시퀀스 길이가 일치하지 않는 경우, 패딩 추가 또는 잘라내기로 길이를 맞추는 전처리 수행

핵심 구조 : Seq2Seq

- 다대다 구조는 Seq2Seq(시퀀스-시퀀스)로 구현됨.
 - 인코더(Encoder): 입력 시퀀스를 처리 → 고정 크기의 벡터로 압축
 - 디코더(Decoder): 인코더의 벡터를 입력받아 출력 시퀀스 생성

양방향 순환 신경망 (Bidirectional Recurrent Neural Network, BiRNN)

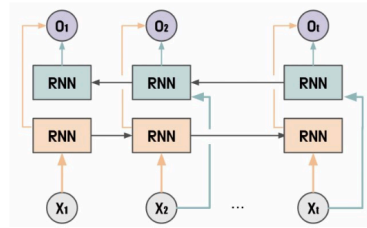


그림 6.16 양방향 순환 신경망 구조

- 기본 RNN은 이전 시점($t-1$)의 정보만 활용
- BiRNN은 이전 시점($t-1$)과 이후 시점($t+1$)의 은닉 상태를 모두 활용

왜 필요한가?

- "인생은 B와 _ 사이의 C다."에서 빈칸을 예측할 때, 앞의 'B와'만으로는 부족하지만 뒤의 '사이의 C이다'까지 보면 'D'임을 알 수 있음
- 즉, 문맥은 앞뒤 모두에서 옴

동작 방식

- 순방향: 입력을 앞에서 뒤로 처리
- 역방향: 입력을 뒤에서 앞으로 처리
- 두 방향의 은닉 상태를 합쳐 현재 시점의 출력값 계산

장점

- 이전 시점 정보만 쓰는 단방향 RNN보다 더 풍부한 문맥 정보를 반영할 수 있음

다중 순환 신경망 (Stacked Recurrent Neural Network)

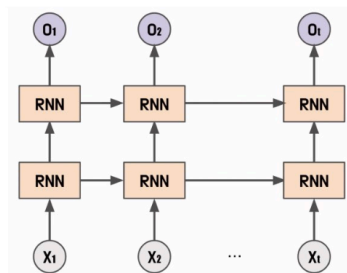


그림 6.17 다중 순환 신경망 구조

- 여러 개의 RNN 층을 쌓아 연결한 구조. 각 층의 출력값이 다음 층의 입력으로 전달됨

왜 쓰는가?

- 단층 RNN보다 더 복잡한 패턴 학습 가능
- 층마다 서로 다른 수준의 특징을 추출함 (다층 퍼셉트론과 같은 원리)

장점

- 데이터의 다양한 특징 추출 가능
- 층이 깊어질수록 더 복잡한 패턴 학습

단점

- 층이 많아질수록 학습 시간 증가

- 기울기 소실(Vanishing Gradient) 문제 발생 가능성 증가

순환 신경망 클래스

```
# 순환 신경망 클래스
# 입력 특성 크기(input_size)와 은닉 상태 크기(hidden_size)를 입력해 순환 신경망 구성 가능
rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1, # 계층 수(RNN의 층수). 2 이상이면 다중 순환 신경망
    nonlinearity="tanh", # 활성화 함수(nonlinearity). 순환 신경망에서 사용되는
    활성화함수 종류(tanh, relu) 설정
    bias=False, # 편향 값 사용 유무
    batch_first=True, # 입력 배치 크기를 첫 번째 차원으로 사용할지 여부
    dropout=0, # dropout 확률 설정
    bidirectional=False # RNN 구조가 양방향으로 처리할지 여부
)
```

```
# 양방향 다층 신경망
import torch
from torch import nn

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape)
print(hidden.shape)
```

▼ 실행 결과

```
torch.Size([4, 6, 512])  
torch.Size([6, 4, 256])
```

장단기 메모리 (Long Short-Term Memory, LSTM)

등장 배경 : 기본 RNN의 두 가지 문제를 해결하기 위해 1997년 제안됨

- 장기 의존성 문제: 시퀀스가 길어질수록 먼 과거 정보가 잘 전달되지 않음
- 기울기 소실/폭주: 역전파 과정에서 기울기가 사라지거나 폭발함

핵심 추가 구조

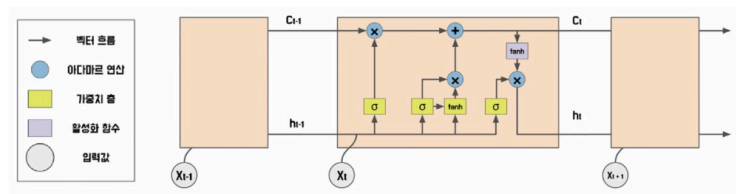


그림 6.18 장단기 메모리 구조

- RNN에 메모리 셀(Cell state)과 3개의 게이트를 추가해 위 문제를 해결.
- 셀 상태(Cell state): 정보를 장기적으로 저장하는 메모리 역할. 망각/출력 게이트에 의해 제어됨.

3개의 게이트

- 망각 게이트(Forget gate): 이전 셀 상태에서 어떤 정보를 삭제할지 결정. 현재 입력 + 이전 은닉 상태를 받아 삭제 비율 결정.
- 입력 게이트(Input gate): 새로운 정보를 얼마나 추가할지 결정. 현재 입력 + 이전 은닉 상태를 받아 추가할 정보 결정.
- 출력 게이트(Output gate): 셀 상태에서 어떤 정보를 출력할지 결정. 현재 입력 + 이전 은닉 상태 + 새 정보를 받아 출력값 결정.

장단기 메모리 구조

전체 흐름 : 메모리 셀(C_t)이 장기 기억을 담당하고, 은닉 상태(h_t)가 단기 출력을 담당. 3개의 게이트가 정보 흐름을 제어함.

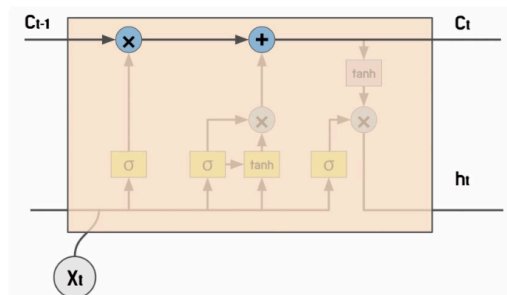


그림 6.19 장단기 메모리 셀 상태

1. 망각 게이트 (Forget Gate)

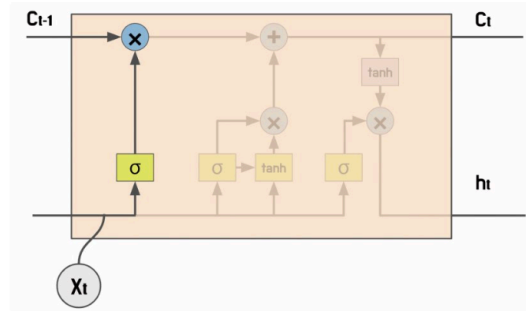


그림 6.20 장단기 메모리의 망각 게이트

- 이전 메모리 셀에서 어떤 정보를 버릴지 결정
- $f_t = \sigma(W_x^{(f)} x_t + W_h^{(f)} h_{t-1} + b^{(f)})$
 - 출력값 0~1 (시그모이드)
 - 1에 가까울수록 정보 유지, 0에 가까울수록 삭제

2. 기억 게이트 (Input Gate)

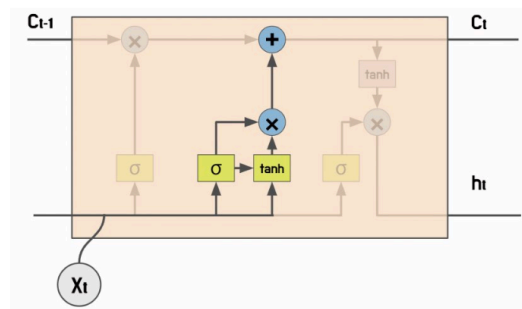


그림 6.21 장단기 메모리의 기억 게이트

- 새로운 정보를 얼마나 추가할지 결정. 두 연산으로 구성
- $g_t = \tanh(W_x^{(g)} x_t + W_h^{(g)} h_{t-1} + b^{(g)})$
- $i_t = \text{sigmoid}(W_x^{(i)} x_t + W_h^{(i)} h_{t-1} + b^{(i)})$

3. 메모리 셀 업데이트

- 망각 게이트 + 기억 게이트 결과를 합산해 현재 메모리 셀 계산
- $c_t = f_t \odot c_{t-1} + g_t \odot i_t$
 - $f_t \odot c_{t-1}$: 이전 기억 중 유지할 부분
 - $g_t \odot i_t$: 새로 추가할 부분
 - $\odot$: 아다마르 곱 (원소별 곱셈)

4. 출력 게이트 (Output Gate)

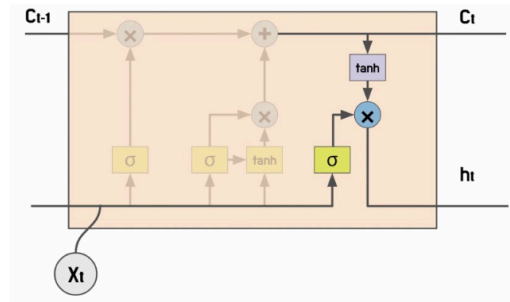


그림 6.22 장단기 메모리의 출력 게이트

- 메모리 셀에서 어떤 정보를 출력할지 결정
- $o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$

5. 은닉 상태 갱신

- $h_t = o_t \odot \tanh(c_t)$
 - o_t : 출력 게이트 (0~1)
 - $\tanh(c_t)$: 메모리 셀을 -1 ~ 1로 정규화
 - 두 값의 아다마르 곱 → 이전 은닉 상태의 영향도 제어

장단기 메모리 클래스

```
# 장단기 메모리 클래스
# LSTM은 활성화함수(nonlinearity) 사용 X
lstm = torch.nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size=0 # 투사 크기. LSTM 메모리 계층의 출력에 대한 선형 투사(Linear Projection) 크기 결정. hidden_size보다 작게 설정
)
```

```
# 양방향 다층 장단기 메모리
import torch
from torch import nn

input_size = 128
output_size = 256
num_layers = 3
bidirectional = True
proj_size = 64

model = nn.LSTM(
```

```

        input_size=input_size,
        hidden_size=output_size,
        num_layers=num_layers,
        batch_first=True,
        bidirectional=bidirectional,
        proj_size=proj_size,
    )

    batch_size = 4
    sequence_len = 6

    inputs = torch.randn(batch_size, sequence_len, input_size)
    h_0 = torch.rand(
        num_layers * (int(bidirectional) + 1),
        batch_size,
        proj_size if proj_size > 0 else output_size,
    )
    c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

    outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

    print(outputs.shape)
    print(h_n.shape)
    print(c_n.shape)

```

▼ 실행 결과

```

torch.Size([4, 6, 128])
torch.Size([6, 4, 64])
torch.Size([6, 4, 256])

```

모델 실습

RNN / LSTM 활용해 문장 긍/부정 분류 모델 학습하기

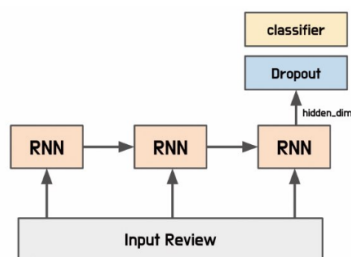


그림 6.23 긍/부정 분류 모델 구조

```

# 문장 분류 모델
from torch import nn

```



```

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm"
    ):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )
        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        if bidirectional:
            self.classifier = nn.Linear(hidden_dim * 2, 1)
        else:
            self.classifier = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        output, _ = self.model(embeddings)
        last_output = output[:, -1, :]

```

```

last_output = self.dropout(last_output)
logits = self.classifier(last_output)
return logits

```

```

# 데이터셋 불러오기
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))

```

▼ 실행 결과

```

...
| 12447 | 잔잔 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
Training Data Size : 45000
Testing Data Size : 5000

```

```

# 데이터 토큰화 및 단어 사전 구축
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"]) # 문장 길이 맞추기 위해 <pad> 추가
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

```

```
print(vocab[:10])
print(len(vocab))
```

▼ 실행결과

```
['<pad>', '<unk>', '.', '이', '영화', '의', '...', '가', '에', '...']
5002
```

```
# 정수 인코딩 및 패딩
import numpy as np

def pad_sequences(sequences, max_length, pad_value): # 너무 긴 문장은 최대 길
    이로 줄이고, 작은 길이는 최대 길이와 동일한 크기로 변환
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in tra
in_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in tes
t_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])
```

▼ 실행 결과

```
[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]
```

```
# 데이터로더 적용
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

```
# 손실 함수와 최적화 함수 정의
from torch import optim

n_vocab = len(token_to_id)
hidden_dim = 64
embedding_dim = 128
n_layers = 2

device = "mps" if torch.backends.mps.is_available() else "cpu"
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim, n
    _layers=n_layers
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device) # BCEWithLogitsLoss : BCELos
s 클래스와 Sigmoid 클래스가 결합된 형태
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001) # RMSProp : 모
든 기울기를 누적하지 않고, 지수 가중 이동 평균(Exponentially Weighted Moving Aver
age, EWMA) 사용해 학습률 조절
```

```
# 모델 학습 및 테스트
def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
```

```

        losses.append(loss.item())

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if step % interval == 0:
        print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits) > .5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(correct
s)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

▼ 실행 결과

```

Train Loss 0 : 0.6942818760871887
Train Loss 500 : 0.6939886836948509
Train Loss 1000 : 0.691114768103048
Train Loss 1500 : 0.681179853894566
Train Loss 2000 : 0.6718297702768098
Train Loss 2500 : 0.664394144700175
Val Loss : 0.6013040644482682, Val Accuracy : 0.6678
Train Loss 0 : 0.5670215487480164
Train Loss 500 : 0.5891966256433856
Train Loss 1000 : 0.5848045533115451
Train Loss 1500 : 0.5742065752847126
Train Loss 2000 : 0.572667495555606
Train Loss 2500 : 0.5700728941897972
Val Loss : 0.545375423119091, Val Accuracy : 0.7362
Train Loss 0 : 0.5273195505142212
Train Loss 500 : 0.4935545513551392
Train Loss 1000 : 0.4877001710764535
Train Loss 1500 : 0.4753848764347998
Train Loss 2000 : 0.46752764537059205
Train Loss 2500 : 0.45962207796858673
Val Loss : 0.43948049357714364, Val Accuracy : 0.7906
Train Loss 0 : 0.3198780417442322
Train Loss 500 : 0.3832911313204708
Train Loss 1000 : 0.38955646784572334
Train Loss 1500 : 0.3896253826835011
...
Train Loss 1500 : 0.3442719578122513
Train Loss 2000 : 0.3431078413690346
Train Loss 2500 : 0.3412974257795203
Val Loss : 0.40453972750768874, Val Accuracy : 0.8216

```

```

# 학습된 모델로부터 임베딩 추출
token_to_embedding = dict()
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()

for word, emb in zip(vocab, embedding_matrix):
    token_to_embedding[word] = emb

token = vocab[1000]
print(token, token_to_embedding[token])

```

▼ 실행 결과

```

보고싶다 [-0.5988024  0.9521534  0.88027793  2.176839  1.2718045 -0.985332
 1.0976859  0.5824067 -0.29054013 -0.738463  0.09776477  0.44202733
 0.19909525 -1.8078574  0.6042558 -1.6221704  1.0816137 -0.4430676
-0.04259913  0.68471867  0.44339374  0.83428794  2.978927  1.6817777
-0.60979414  0.04855198  1.0779232 -0.8695096  0.89842194 -0.09096065
 0.11334154  0.6659418 -0.23292296 -1.5131161 -0.67398  0.62993914
-1.0296423  0.33177245 -0.58650035 -0.0818486  0.51450646  0.1477467
 1.3412278 -0.13525425  0.56022197  0.20898616  1.6585898  0.7172514
 0.58321404  2.4854639 -0.23563597  0.60478956 -2.0695803 -1.5799614
-0.33183232  0.8665989  0.03166395 -1.0117462  1.6737714 -1.3130771
 0.68908304  1.0471203  1.3326067  0.37814862  0.2767778 -0.7316114
 0.36340067  1.9225087 -1.5530314 -0.2248208 -0.6166332 -0.10490787
 1.0363888 -0.09873985  0.7795527 -0.34922415  0.39081594  0.834382
 0.3959935  0.3093479  1.3445561 -1.5483539 -1.6371776  0.6815365
-1.5897181 -0.21040103  0.7286222  0.3827533  1.5508964 -0.651729
 0.7584359 -1.4557339 -2.2115045  0.19091359  1.0267198  0.6070805
 0.2663974 -0.355746  0.51765925 -1.0332891  0.15909137  0.8434846
-0.5651441  1.1475368  0.5154817  1.0281855 -0.7504779  0.2645516
-0.08003215  0.6637621 -1.6139188 -0.7961587 -0.02834903  0.01260137
-3.1619134 -1.3735522  1.5924199 -1.1367772 -3.0899887  0.90111095
-1.2641736  0.9033164 -0.8335668  1.2618759 -0.76544297  1.5015159
 1.8167208  0.1689868 ]

```

```

# 사전 학습된 모델로 임베딩 계층 초기화
from gensim.models import Word2Vec

word2vec = Word2Vec.load("../models/word2vec.model")
init_embeddings = np.zeros((n_vocab, embedding_dim))

for index, token in id_to_token.items():
    if token not in ["<pad>", "<unk>"]:
        if token in word2vec.wv: # 이 줄 추가
            init_embeddings[index] = word2vec.wv[token]

embedding_layer = nn.Embedding.from_pretrained(
    torch.tensor(init_embeddings, dtype=torch.float32)
)

```

```

# 사전 학습된 임베딩 계층 적용
class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm",
        pretrained_embedding=None
    ):
        super().__init__()

        self.embedding = nn.Embedding(

```

```

        num_embeddings=n_vocab,
        embedding_dim=embedding_dim,
        padding_idx=0
    )
    if model_type == "rnn":
        self.model = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            bidirectional=bidirectional,
            dropout=dropout,
            batch_first=True,
        )
    elif model_type == "lstm":
        self.model = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            bidirectional=bidirectional,
            dropout=dropout,
            batch_first=True,
        )

    if bidirectional:
        self.classifier = nn.Linear(hidden_dim * 2, 1)
    else:
        self.classifier = nn.Linear(hidden_dim, 1)
    self.dropout = nn.Dropout(dropout)

    if pretrained_embedding is not None:
        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
    else:
        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        output, _ = self.model(embeddings)
        last_output = output[:, -1, :]
        last_output = self.dropout(last_output)
        logits = self.classifier(last_output)
        return logits

```



```
# 사전 학습된 임베딩을 사용한 모델 학습
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim,
    n_layers=n_layers, pretrained_embedding=init_embeddings
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)
```

▼ 실행 결과

```
Train Loss 0 : 0.6955875158309937
Train Loss 500 : 0.6940808579355419
Train Loss 1000 : 0.6932309921923931
Train Loss 1500 : 0.690803829627701
Train Loss 2000 : 0.6903715668649211
Train Loss 2500 : 0.6910979660784231
Val Loss : 0.6892091395755926, Val Accuracy : 0.5284
Train Loss 0 : 0.7176898121833801
Train Loss 500 : 0.687129848017664
Train Loss 1000 : 0.6882573206584294
Train Loss 1500 : 0.6863436407839593
Train Loss 2000 : 0.6822881453904672
Train Loss 2500 : 0.6786381721806403
Val Loss : 0.663050505680779, Val Accuracy : 0.6058
Train Loss 0 : 0.645942747592926
Train Loss 500 : 0.6578854481855076
Train Loss 1000 : 0.6563061067869851
Train Loss 1500 : 0.6535676505508461
Train Loss 2000 : 0.6515718378435665
Train Loss 2500 : 0.6475710731084611
Val Loss : 0.650943220042573, Val Accuracy : 0.6254
Train Loss 0 : 0.6774590015411377
Train Loss 500 : 0.6430164505383688
Train Loss 1000 : 0.6372791067227259
Train Loss 1500 : 0.6352103229644377
...
Train Loss 1500 : 0.6219106618163587
Train Loss 2000 : 0.6208696282696331
Train Loss 2500 : 0.6199002175724826
Val Loss : 0.6251876921699451, Val Accuracy : 0.6454
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

합성곱 신경망 (Convolutional Neural Network, CNN)

- 입력 데이터의 지역적 특징을 추출하는 데 특화된 신경망. 합성곱(Convolution) 연산을 사용

핵심 특징

- 지역 특징(Local Features): 특정 영역 내에서만 연산 → 지역 패턴 효과적으로 추출

- 전역 특징(Global Features): 여러 지역 특징을 조합 → 전체적인 패턴 파악

RNN 대비 장점 (NLP에서)

- RNN은 순서대로 처리해야 해서 병렬 처리 불가
- CNN은 입력 길이와 무관하게 병렬 처리 가능
- 가중치 수가 적어 깊은 신경망 구성이 용이

NLP에 적용된 배경

- 원래 컴퓨터비전용으로 개발됐지만, 2014년 사전 학습된 임베딩과 CNN을 결합한 문장 분류 모델이 등장하면서 자연어 처리에도 활발히 사용되기 시작.

합성곱 계층

- 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층. 이미지, 음성 등 고차원 데이터 처리에 주로 사용.

핵심 특성

- 매개변수 공유: 입력 데이터의 모든 위치에서 동일한 필터를 사용 → 학습해야 할 매개변수 수 감소 → 과대적합 방지
- 위치 불변성: 특징이 이미지 내 어느 위치에 있어도 동일하게 추출 가능
- 계층을 쌓을수록 합성곱 계층이 많아질수록 모델 복잡도 증가 → 더 다양하고 추상적인 특징 학습 가능.

필터 (Filter)

- 합성곱 연산에 사용되는 작은 크기의 정방형 행렬
- 커널(Kernel) 또는 윈도우(Window)라고도 불림
- 일반적으로 3×3, 5×5 크기

동작 방식

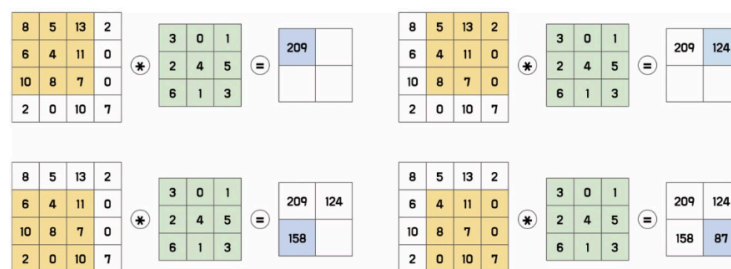


그림 6.24 특징 맵 추출 과정

1. 필터를 입력 이미지의 왼쪽 상단부터 오른쪽 하단으로 이동
2. 필터와 대응하는 입력 영역을 원소별로 곱한 후 모두 더함
3. 그 결과를 특징 맵(Feature Map)의 한 원소로 저장
4. 이 과정을 반복해 특징 맵 전체를 생성

e.g. 4×4 입력에 3×3 필터를 1칸씩 이동 → 2×2 특징 맵 생성

핵심 특성

- 필터의 가중치는 학습을 통해 갱신됨
- 하나의 필터가 전체 입력에서 공용 가중치로 재사용 → 위치와 무관하게 동일한 패턴 학습 가능
- 여러 개의 필터를 사용해 다양한 특징 동시 추출

패딩 (Padding)

문제 : 특징 맵 크기 감소

- 합성곱 연산을 수행하면 출력(특징 맵) 크기가 입력보다 작아짐. (e.g. 4×4 입력 + 3×3 필터 → 2×2 특징 맵)
- 두 가지 부작용 :
 - 신경망을 깊게 쌓을수록 특징 맵이 너무 작아짐
 - 가장자리 픽셀은 연산에 적게 참여 → 가장자리 정보 손실

해결책 : 패딩

- 입력 데이터의 가장자리에 특정 값을 덧붙여 크기를 유지
- 제로 패딩(Zero Padding): 가장자리에 0을 추가하는 방식 (가장 일반적)

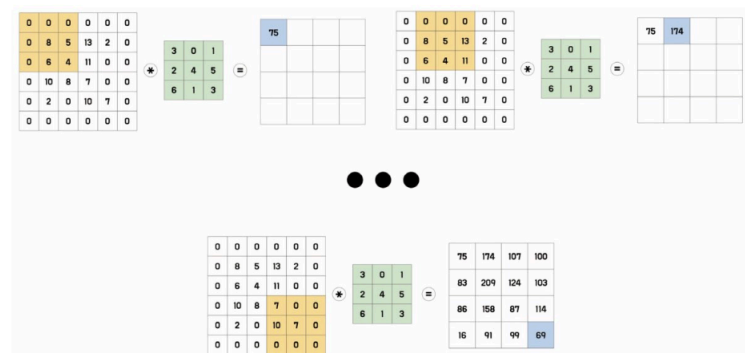


그림 6.25 제로 패딩

- e.g. 4×4 입력에 제로 패딩 → 6×6으로 확장 후 3×3 필터 적용 → 4×4 특징 맵 (입력과 동일한 크기 유지)

효과

- 입력과 출력 크기를 동일하게 유지 가능
- 작은 필터로도 이미지 전체 정보 반영 가능
- 더 깊은 네트워크 구성 가능

간격 (Stride)

- 필터가 한 번에 이동하는 크기
 - Stride = 1: 한 픽셀씩 이동 → 출력 크기 = 입력과 동일하거나 유사
 - Stride ≥ 2: 더 큰 간격으로 이동 → 출력 크기 < 입력 크기

Stride 크기의 효과

- 작은 Stride: 공간 정보 보존, 출력 크기 크게 유지
- 큰 Stride: 공간 정보 감소, 출력 크기 축소, 학습해야 할 매개변수 수 감소 → 모델 복잡도 감소, 과대적합 방지

공간 정보란?

- 픽셀 간의 상대적인 위치나 거리 정보. 이미지의 형태와 구조를 나타내는 데 중요한 역할을 함

채널

- 입력 데이터와 필터가 3차원으로 구성될 때, 같은 위치의 값끼리 연산이 수행되는 차원. 공간 정보를 유지하면서 추출되는 특징을 확장하는 역할
- e.g. RGB 이미지
R, G, B 각 채널마다 동일한 필터가 존재 → 각 채널의 정보를 따로 추출해 특징 맵 생성 → 특징 맵 개수 = 채널 수

채널 수의 효과

- 많을수록: 학습할 수 있는 특징의 다양성 증가 → 모델의 표현력(Representational Power) 향상, 더 복잡한 문제 해결 가능
- 적을수록: 매개변수 수 감소 → 학습 시간과 메모리 사용량 절약

트레이드오프

- 채널이 많을수록 매개변수도 많아져 학습 시간·메모리 증가, 과대적합 위험 증가
- 모델 구조와 목적에 맞게 채널 수를 적절히 설정해야 함

팽창 (Dilation)

- 필터와 입력 데이터 사이에 간격을 두어, 필터 크기를 키우지 않고도 더 넓은 범위의 입력을 고려하는 기법

동작 방식

- dilation=1: 일반 합성곱과 동일 (간격 없음)
- dilation=2: 한 칸씩 건너뛰며 연산 → 더 넓은 범위 커버
- 값이 커질수록 필터가 바라보는 수용 영역(Receptive Field)이 넓어짐

장점

- 필터 크기를 늘리지 않아도 넓은 영역 고려 가능
- 매개변수 수를 줄여 메모리 사용량 절약
- 더 깊고 복잡한 모델 구성 가능

단점

- 바라봐야 할 범위가 커지므로 연산량이 증가할 수 있음
- dilation이 너무 크면 인접 픽셀을 고려하지 않게 되어 공간 정보 손실 → 특징 추출 효과 저하

합성곱 계층 클래스

```
# 클래스 구조
conv = torch.nn.Conv2d(
    in_channels,      # 입력 채널 수
    out_channels,     # 출력 채널 수
    kernel_size,      # 필터 크기
    stride=1,         # 이동 간격
    padding=0,        # 패딩 크기
    dilation=1,       # 팽창 크기
```

```

groups=1,          # 채널 그룹 수
bias=True,         # 편향 포함 여부
padding_mode="zeros" # 패딩 방식
)

```

주요 파라미터 설명

- groups: 입력/출력 채널을 그룹으로 나눠 각각 합성곱 수행. 2 이상이면 매개변수 수 감소. 단, 입력/출력 채널 수가 groups의 배수여야 함
- padding_mode :
 - zeros: 제로 패딩 (기본값)
 - reflect: 가장자리를 거울처럼 반사해 채움
 - replicate: 가장자리 값을 복사해 채움

출력 크기 계산 공식

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$

$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

활성화 맵 (Activation Map)

- 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻은 출력값
- CNN에서는 일반적으로 ReLU를 사용
 - ReLU: 값 > 0이면 그대로 출력, 값 ≤ 0이면 0 출력

왜 활성화 함수가 필요한가?

- 활성화 함수 없이는 합성곱 연산 결과가 그대로 다음 계층으로 전달되어 모델이 선형 결합만 수행하게 됨 → 복잡한 패턴이나 추상적인 특징 학습 불가
- 활성화 함수를 적용하면 비선형성이 추가되어 다양한 추상적 특징 학습 가능

동작 흐름

- 합성곱 연산 → 특징 맵 → 활성화 함수 적용 → 활성화 맵 → 다음 계층 입력
- 이 과정을 여러 번 반복할수록 더 추상적인 특징 학습

활성화 맵의 활용

- 시각화하면 모델이 입력 이미지의 어떤 부분에 반응하는지, 어떤 특징을 학습하는지 파악 가능 → 모델 해석에 유용

풀링

- 특징 맵의 크기를 줄여 연산량을 감소시키고 정보를 압축하는 연산

- 합성곱 계층 다음에 적용

두 가지 방식

- 최댓값 풀링(Max Pooling): 필터 내 원숫값 중 가장 큰 값 선택. 가장 두드러진 특징 보존

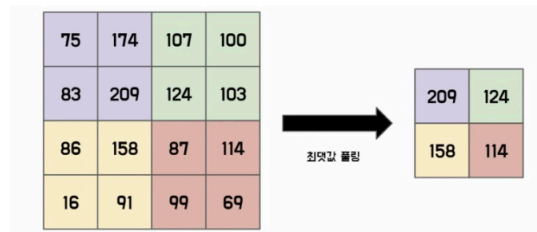


그림 6.27 최댓값 풀링

- 평균값 풀링(Average Pooling): 필터 내 원숫값의 평균 사용. 전체적인 특징을 부드럽게 압축.

장점

- 공간 크기 감소 → 연산량·메모리 절약
- 특징 위치가 약간 변해도 인근 영역 연산으로 공간 정보 유지

단점

- 위치 정보 일부 손실 → 세밀한 위치 정보가 필요한 작업에서는 성능 저하 가능
- 최근에는 풀링 대신 합성곱 계층의 stride를 높여 크기를 줄이는 방식을 더 많이 사용

파이토치 클래스

```
# 최댓값 풀링
torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1)

# 평균값 풀링
torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, count_include_pad=
True)
# count_include_pad=True: 패딩 영역의 0도 평균 계산에 포함
```

출력 크기 공식

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층 (Fully Connected Layer, FC)

- 각 입력 노드가 모든 출력 노드와 연결된 층. 입력과 출력 간의 모든 가능한 관계를 학습한다.

CNN에서의 역할

- conv + pooling 층을 거친 3D 특징 맵 → flatten으로 1D 벡터화 → FC 층 입력
- 전체 입력과 가중치의 내적 연산으로 출력값 계산

- 공간 정보는 이 시점에서 무시됨 (이미 conv에서 추출 완료)
- 최종 출력에 softmax / sigmoid 적용 → 분류 수행

CNN 전체 파이프라인

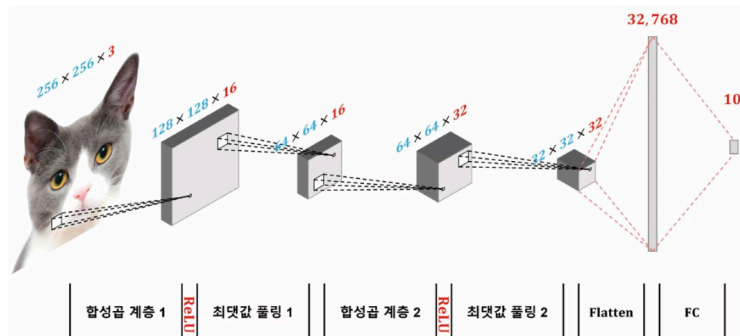


그림 6.28 합성곱 모델 구조 시각화

256x256x3 입력
 → Conv1 + ReLU → MaxPool1 (128x128x16 → 64x64x16)
 → Conv2 + ReLU → MaxPool2 (64x64x32 → 32x32x32)
 → Flatten (32,768 벡터)
 → FC (10 출력)

핵심 포인트

- conv 층 = 공간 특징 추출 담당
- FC 층 = 추출된 특징으로 최종 분류 담당
- 출력 노드 수 조절로 모델 용량 조정 가능

```
# 합성곱 모델
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16, out_channels=32, kernel_size=3, stride=1,
```

```
padding=1
),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2),
)

self.fc = nn.Linear(32 * 32 * 32, 10)

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    x = torch.flatten(x)
    x = self.fc(x)
    return x
```

모델 실습

텍스트용 CNN : 1차원 합성곱 (1D Convolution)

왜 1D인가?

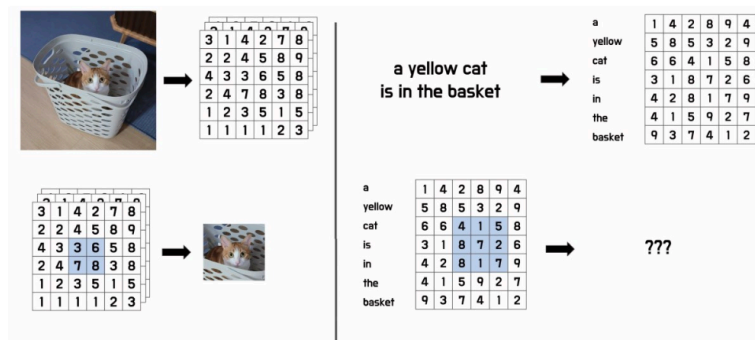


그림 6.29 이미지 데이터와 텍스트 데이터의 2차원 합성곱

- 텍스트 임베딩은 행 순서(단어 순서)는 의미가 있지만 열 방향(임베딩 차원 내)은 위치 의미가 없다. 따라서 필터가 수직 방향(단어 방향)으로만 이동하는 1D conv를 써야 한다.

입력 구조

문장 = [단어 수 × 임베딩 차원]
 예: 7단어 × 6차원 → (7, 6) 행렬

- 필터 너비 = 임베딩 차원과 동일하게 고정, 높이만 조절 (필터 크기 = n-gram 크기)

필터 크기의 의미

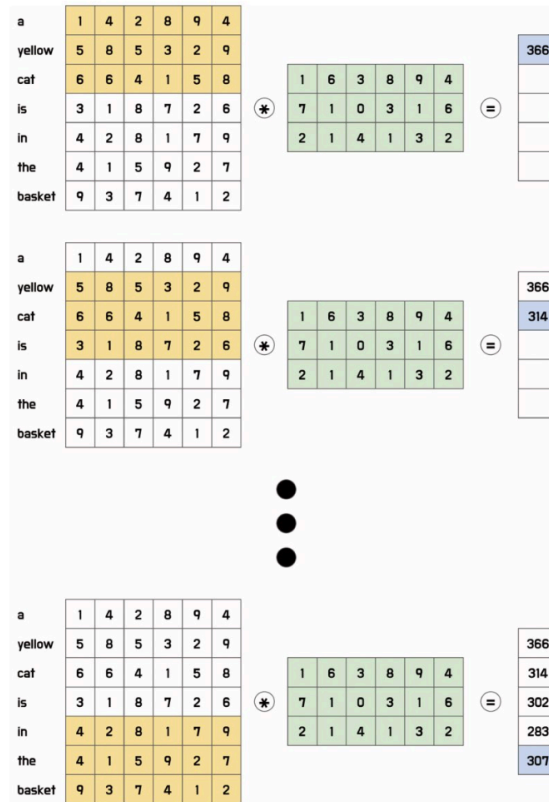


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

- 필터 크기 3 → 인접 3개 토큰을 보며 연산 (3-gram과 유사)
- 필터 크기 2, 3, 4 등 다양하게 사용 → 다양한 길이의 문맥 패턴 포착

전체 파이프라인

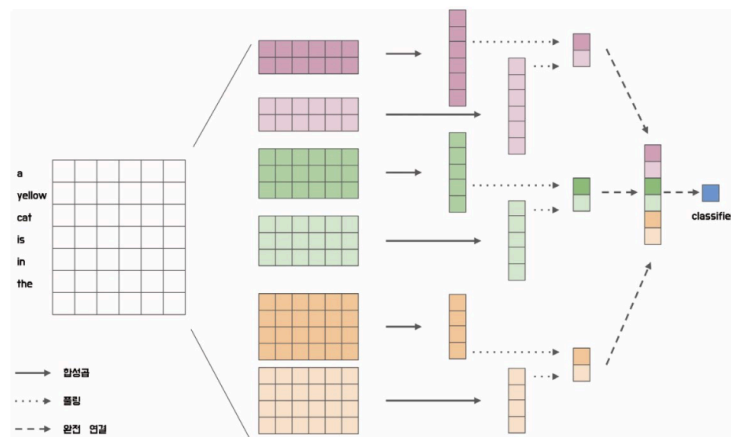


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산

텍스트 임베딩 행렬

- 크기 2, 3, 4 필터 각 2개씩 적용 (1D Conv)
- 각 필터마다 1D 벡터 출력
- Pooling → 스칼라값
- 전체 concatenate → 특징 벡터
- FC → classifier

핵심 포인트

- 이미지: 2D conv (공간 전방향 이동)
- 텍스트: 1D conv (단어 방향으로만 이동)
- 다양한 크기의 필터를 병렬로 써서 다양한 n-gram 정보를 동시에 포착
- 사전 학습된 임베딩(Word2Vec 등) 사용이 일반적 → 안 쓰면 성능 저하 가능

```
# 합성곱 기반 문장 분류 모델 정의
import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(self, pretrained_embedding, filter_sizes, max_length, dropout=0.5):
        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
        embedding_dim = self.embedding.weight.shape[1]

        conv = []
        for size in filter_sizes:
            conv.append(
                nn.Sequential(
                    nn.Conv1d(
                        in_channels=embedding_dim,
                        out_channels=1,
                        kernel_size=size
                    ),
                    nn.ReLU(),
                    nn.MaxPool1d(kernel_size=max_length-size-1),
                )
            )
        self.conv_filters = nn.ModuleList(conv)

        output_size = len(filter_sizes)
        self.pre_classifier = nn.Linear(output_size, output_size)
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(output_size, 1)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        embeddings = embeddings.permute(0, 2, 1)

        conv_outputs = [conv(embeddings) for conv in self.conv_filters]
        concat_outputs = torch.cat([conv.squeeze(-1) for conv in conv_outp
```

```

uts], dim=1)

        logits = self.pre_classifier(concat_outputs)
        logits = self.dropout(logits)
        logits = self.classifier(logits)
        return logits

```

```

# 합성곱 신경망 분류 모델 학습
from torch import optim

n_vocab=len(token_to_id)
hidden_dim=64
embedding_dim=128
n_layers=2

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
filter_sizes = [3, 3, 4, 4, 5, 5]
classifier = SentenceClassifier(
    pretrained_embedding=init_embeddings,
    filter_sizes=filter_sizes,
    max_length=max_length
).to(device)

criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.Adam(classifier.parameters(), lr=0.001)

```

```

# 데이터셋 불러오기
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))

```

▼ 실행 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 description 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39

Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.

The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.

[^1]: http://ai.stanford.edu/~amaas/data/sentiment/

# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
...
| 12447 | 찬찬 격동
| 39489 | 오랜만에 찾은 주말의 영화의 보석
Training Data Size : 45000
Testing Data Size : 5000
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```
# 데이터 토큰화 및 단어 사전 구축
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"]) # 문장 길이 맞추기 위해 <pad> 추가
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

▼ 실행 결과

```
['<pad>', '<unk>', '.', '이', '영화', '의', '...', '가', '에', '...']
5002
```

```

# 정수 인코딩 및 패딩
import numpy as np

def pad_sequences(sequences, max_length, pad_value): # 너무 긴 문장은 최대 길
    이로 줄이고, 작은 길이는 최대 길이와 동일한 크기로 변환
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in tra
in_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in tes
t_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])

```

▼ 실행 결과

```

[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]

```

```

# 데이터로더 적용
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)

```

```

test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

```

모델 학습 및 테스트

```

def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if step % interval == 0:
            print(f"Train Loss {step} : {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits) > .5
        corrects.extend(
            torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(correct

```

```
s)}}")
```

```
epochs = 5
```

```
interval = 500
```

```
for epoch in range(epochs):
```

```
    train(classifier, train_loader, criterion, optimizer, device, interval)
```

```
    test(classifier, test_loader, criterion, device)
```

▼ 실행 결과

```
Train Loss 0 : 0.7049808502197266
Train Loss 500 : 0.6807672238635446
Train Loss 1000 : 0.6598644902656129
Train Loss 1500 : 0.6472688789053173
Train Loss 2000 : 0.6393669200860995
Train Loss 2500 : 0.6335371304206588
Val Loss : 0.5970501449351875, Val Accuracy : 0.669
Train Loss 0 : 0.5907931327819824
Train Loss 500 : 0.5965058095678836
Train Loss 1000 : 0.5988846679965218
Train Loss 1500 : 0.6000960933653853
Train Loss 2000 : 0.6000064159112832
Train Loss 2500 : 0.5983456621547548
Val Loss : 0.5899417880244149, Val Accuracy : 0.6648
Train Loss 0 : 0.6278339624404907
Train Loss 500 : 0.5777213537764406
Train Loss 1000 : 0.5845607613231038
Train Loss 1500 : 0.5854598088196165
Train Loss 2000 : 0.587392166964356
Train Loss 2500 : 0.5885945666627568
Val Loss : 0.5901025545101958, Val Accuracy : 0.671
Train Loss 0 : 0.5281313061714172
Train Loss 500 : 0.5808150870595388
Train Loss 1000 : 0.5808097034186631
Train Loss 1500 : 0.5800262329222599
...
Train Loss 1500 : 0.574192940513743
Train Loss 2000 : 0.5747080510524081
Train Loss 2500 : 0.5747899948382845
Val Loss : 0.5817976813918104, Val Accuracy : 0.677
```